

Universidad Distrital Francisco José de Caldas

Computer Engineering Program — School of Engineering

Systems Analysis & Design — Semester 2026-I

Eng. Carlos Andrés Sierra, M.Sc.

Community Security Alert System (CSAS)

Course Project — Comprehensive Technical Report

Workshops No. 1, 2, 3 & 4 + Functional Prototype Implementation

Felipe Jose Garzon Herrera

Code: 20251020132

Juan Esteban Quintero Gordillo

Code: 20251020137

Henry Samuel Garrido Medina

Code: 20251020125

Gabriel Mateo Cusba Marin

Code: 20251020128

Bogotá D.C., Colombia — March–June 2026

Submitted in partial fulfillment of the requirements for the

Systems Analysis & Design course, Semester 2026-I.

0 ABSTRACT

Campus security at Universidad Distrital Francisco José de Caldas is impaired by three compounding problems identified through systematic primary data collection: a persistent underreporting culture attributable to the high friction of current reporting channels, a spatial and temporal concentration of incidents during evening and night hours in specific campus zones, and the absence of a centralized real-time incident management tool that forces security staff to rely on phone calls and walk-up reports, resulting in average response delays of approximately 11.4 minutes. This report presents the Community Security Alert System (CSAS) across its complete systems engineering lifecycle, covering four workshops and a functional prototype. The microservices-based, event-driven architecture integrates a hybrid AI-and-human verification pipeline, geofenced alert dispatch with zone-weighted and time-weighted priority logic, and a multi-channel notification engine. The proposed design targets a reduction of emergency response time to under 5 minutes, an increase of incident recording completeness from 41% to above 85%, and a community adoption rate of at least 60%. Robust engineering principles (Workshop 3) and discrete event plus behavioral simulation (Workshop 4) validated the architecture and identified the verification threshold ($\theta \geq 3$) as the most critical control parameter. A functional web prototype was developed in React as a proof-of-concept demonstrating the core workflows — incident reporting, AI-assisted verification, geofenced dispatch, analytics, and the simulation console — traceable to all requirements defined across the workshop sequence.

Keywords: *campus security, crowd-sourced incident reporting, microservices architecture, real-time notifications, geofenced alerts, systems analysis, systems design, risk management, simulation, functional prototype, React, Universidad Distrital.*

1 INTRODUCTION

Campus safety is a persistent challenge for universities in urban Latin American environments. At Universidad Distrital Francisco José de Caldas, a public university located in Bogotá D.C., the absence of a centralized digital reporting tool means that security incidents are communicated through informal channels — phone calls, WhatsApp groups, and walk-up reports to security guards — introducing delays, data loss, and a systemic inability to detect spatial or temporal patterns. The average time from incident occurrence to security team notification has been measured at approximately 11.4 minutes, a figure that represents a significant risk during fast-developing emergencies such as assaults or medical events.

Several platforms have attempted to address similar challenges in the Bogotá metropolitan area. The Policía Nacional's Vigila App integrates directly with law enforcement but relies on the user placing a phone call, producing slow response cycles. Neighborhood WhatsApp networks achieve high adoption rates due to their familiarity but operate without any verification mechanism, systematically generating false alarms and panic. SIURE UD, the university's own internal incident reporting system, provides an existing institutional process but operates exclusively through email and phone, offering no mobile interface or real-time notification capability. These precedents establish a clear gap: no existing platform combines low-friction mobile reporting, real-time geofenced notification, and verified alert quality in an institutional campus context.

The Community Security Alert System (CSAS) is proposed to fill this gap. This report integrates four phases of the project: Workshop No. 1 (empirical systems analysis), Workshop No. 2 (formal systems architecture design), Workshop No. 3 (robust engineering and project management), and Workshop No. 4 (simulation and validation). Each phase builds directly on the findings of the previous one, ensuring that every architectural decision is traceable to real empirical data and that the design is both technically sound and operationally validated before implementation.

2 LITERATURE REVIEW

The theoretical foundation of this project draws from established systems thinking and engineering frameworks. Checkland's soft systems methodology provides the conceptual basis for treating campus security as a sociotechnical system in which human behavior, institutional processes, and technology interact in complex, non-linear ways. Meadows offers the analytical vocabulary for understanding the feedback dynamics that govern adoption and alert quality in a crowd-sourced platform: the reinforcing loops that amplify participation and the balancing loops that must be calibrated to prevent alert fatigue.

The ISO/IEC/IEEE 15288:2015 standard provides the life-cycle process framework within which the CSAS analysis and design activities are situated. Creswell and Creswell inform the mixed-methods data collection strategy employed in Workshop No. 1, combining structured interviews, surveys, direct observation, document analysis, benchmarking, and environmental monitoring into a multi-method design that strengthens the validity of findings through methodological triangulation. Bass, Clements, and Kazman ground the design decisions related to modularity, scalability, and separation of concerns that motivate the microservices decomposition. Dragoni et al. provide a contemporary treatment of microservices as an architectural style, validating its suitability for systems with heterogeneous load profiles.

For the robust design phase, ISO/IEC 25010 provides the quality model that frames the reliability, scalability, maintainability, usability, and security objectives. ISO 31000 and PMBOK structure the risk management and project management frameworks applied in Workshop No. 3. Law and Sterman inform the simulation methodology of Workshop No. 4, particularly the use of discrete event simulation and system dynamics for validating sociotechnical system behavior.

3 OBJECTIVES

3.1 General Objective

To design, validate, and document a community-driven, real-time incident reporting and alert system for the Universidad Distrital Francisco José de Caldas campus that demonstrably reduces emergency response times and improves incident recording completeness relative to the current state.

3.2 Specific Objectives

- To characterize the current campus security posture through primary data collection, identifying key gaps, behavioral parameters, and system boundaries (Workshop No. 1).
 - To translate the empirical findings of the systems analysis into a formal set of functional and non-functional requirements explicitly traceable to observed problems.
 - To design a microservices-based architecture that addresses the identified gaps through modular, independently scalable components (Workshop No. 2).
 - To define a hybrid AI-and-human verification pipeline that balances alert latency with alert signal quality.
 - To strengthen the architecture through robust engineering principles, a risk management plan, and a project management framework aligned with ISO/IEC 25010, ISO 31000, and PMBOK (Workshop No. 3).
 - To validate the design through discrete event simulation and behavioral simulation, confirming feasibility under realistic operating conditions (Workshop No. 4).
-

- To specify a technical implementation strategy — including technology stack, implementation phases, and key performance indicators — as a foundation for future development.

4 SCOPE

The CSAS scope is defined by a boundary that separates the platform's direct responsibilities from its operational environment. Table 1 formalizes this boundary. The system covers the campus of Universidad Distrital Francisco José de Caldas and its immediate surrounding streets, and serves six stakeholder groups: students (primary reporters and alert recipients), professors and administrative staff (secondary reporters), campus security staff (verifiers and response coordinators), local authorities (recipients of escalated verified incidents), system administrators (platform maintainers), and university management (consumers of analytics and security reports).

Table 1. System Boundary Definition — CSAS

Inside the System (Scope)	Outside the System (Environment)
Incident reporting and management platform	University physical infrastructure
Crowd-sourced verification logic	Local law enforcement (Police / EMS)
Real-time push notification engine	Public telecommunications and ISP networks
User and incident databases	Non-registered external visitors
Emergency coordination dashboard	Campus security physical hardware (CCTV, etc.)
Analytics and heatmap generation	University academic or administrative systems

5 ASSUMPTIONS

The analysis and design documented in this report rest on three critical assumptions that have not yet been validated under real operating conditions and that represent the primary behavioral and technical risks for the implementation phase.

Willingness to report. It is assumed that students and staff will submit incident reports if the reporting process is made sufficiently simple and low-friction. Workshop No. 1 survey data provides directional support: all participants who had personally experienced a campus incident stated they would use the application. However, this preference expressed under survey conditions has not been confirmed in real operational settings. Simulation results from Workshop No. 4 further challenge this assumption: the behavioral model shows that 12% initial adoption (the observed baseline) is insufficient for the system's reinforcing loop to activate without a structured onboarding campaign.

Verification latency tolerance. The design assumes that the verification mechanism can process incoming reports and produce a verified alert within a 45-second latency budget for standard-severity incidents. Workshop No. 4 simulation confirmed this is achievable under the Optimization scenario (average latency 17.94 s) but not under the Baseline or Failure Mode scenarios. This assumption is designated for validation through load testing in Phase 4.

Mobile connectivity coverage. The system assumes reliable mobile network and WiFi connectivity across all campus zones. The direct observation phase of Workshop No. 1 did not include a systematic assessment of signal coverage. Dead zones in low-connectivity areas could materially degrade alert delivery guarantees and represent a prerequisite risk for infrastructure planning.

6 LIMITATIONS

Several limitations constrain the reach of the conclusions drawn in this report. These are acknowledged not as weaknesses that invalidate the work, but as boundaries that define the scope of the evidence base and establish a research agenda for subsequent phases.

The student survey conducted in Workshop No. 1 (n = 20) is too small for statistical generalization to the full university population. The direct observation phase covered only four campus zones across eight sessions; indoor corridors, cafeterias, and transit areas were not observed. The document analysis was limited by restricted access to institutional security records. The weather correlation finding is based on a one-week window and is insufficient to establish causality.

From Workshop No. 4, the simulation relies on simplified probabilistic rules for AI verification confidence (uniform distribution) rather than a trained model, and on randomly generated GPS coordinates rather than real spatiotemporal incident distributions from Workshop No. 1. No real mobile application was implemented or tested. Human behavior is approximated statistically, not observed. These simulation limitations mean that all performance figures are targets and model-derived estimates, not measured outcomes.

Despite these limitations, the convergence of three independent methods — interviews, surveys, and direct observation — on the same three core problems, combined with the simulation validation of the proposed mitigations, provides sufficient confidence to guide the design and implementation phases.

7 METHODOLOGY

7.1 Systems Analysis Approach (Workshop No. 1)

The systems analysis phase employed a mixed-methods approach that combined six complementary data collection methods to characterize the campus security problem from multiple independent perspectives. Table 2 summarizes the six methods applied.

Table 2. Data Collection Plan — Six Methods Applied to the CSAS

Method	Tool	Sample	Dur.	Objective
Interviews	Structured guide	5 security, 2 admin staff	1 week	Understand current workflows and identify unmet needs.
Surveys	Google Forms	20 students	2 weeks	Assess safety perception, incident experience, and app adoption intent.
Direct observation	Record form	4 campus zones	1 week	Record activity level and suspicious events by time and zone.
Document analysis	Institutional reports	Campus records	3 days	Identify historical incident patterns and cross-validate field findings.
Benchmarking	Comparative analysis	3 apps in Bogotá	1 week	Extract design insights from existing platforms.
Weather monitoring	Weather apps	Teusaquillo locality	1 week	Explore correlation between adverse weather and incident occurrence.

Structured interviews with five security staff members and two administrative staff members revealed three consistent problems: slow communication times, absence of a centralized reporting tool, and the need for faster notifications with exact incident location. The student survey (n = 20) showed that 35% had experienced or witnessed an incident, with all reported events occurring during afternoon and night hours, and 100% of those with incident experience expressing willingness to use the alert application. Direct observation confirmed the parking lot at night as the highest-risk zone, with both suspicious events observed going unreported by bystanders, directly reinforcing the under-reporting hypothesis. Document analysis confirmed that formal records capture far fewer incidents than actually occur. Benchmarking evaluated three platforms in Bogotá (results in Appendix C). Weather monitoring found both suspicious events coincided with rainy nights.

7.2 System Architecture Design (Workshop No. 2)

The systems design phase translated the empirical findings of Workshop No. 1 into a formal architecture using an iterative, requirements-driven approach consistent with the ISO/IEC/IEEE 15288:2015 systems life cycle. The architectural style was selected based on the non-uniform load profiles of the CSAS subsystems and the need for independent scalability, motivating a microservices-based, event-driven decomposition.

The architecture is organized across five layers — Presentation, Application, Service, Data, and Integration. The five core microservices (Incident, Verification, Dispatcher, User, and Analytics) communicate asynchronously through a message broker, ensuring loose coupling between the ingestion and processing stages. The verification pipeline implements a hybrid approach: an AI plausibility scorer performs initial metadata validation, while a human moderator reviews low-confidence reports. High-severity incident types (assault, medical emergency) have a priority bypass path.

7.3 Robust System Design (Workshop No. 3)

Workshop No. 3 strengthened the Workshop No. 2 architecture by binding every component to measurable quality attributes and recognized engineering standards. The four robustness pillars — reliability, scalability, maintainability, and usability — were formalized using ISO/IEC 25010 as the evaluation framework.

A comprehensive risk management plan following ISO 31000 produced a register of ten items spanning technical, security, operational, and project categories, each scored on a probability–impact matrix and matched with specific mitigation, contingency, and monitoring strategies. A project management plan aligned with PMBOK defined four team roles, a six-phase critical path, and a phased deployment strategy comprising pilot, controlled rollout, and full operation phases. Table 8 presents the standards compliance matrix and Table 9 the full risk register.

7.4 System Simulation and Validation (Workshop No. 4)

Workshop No. 4 validated the CSAS design through two complementary simulation approaches implemented in Python: a Discrete Event Simulation (DES) using the SimPy library, and a hybrid behavioral simulation capturing community adoption dynamics and feedback loops. Three operational scenarios were evaluated — Baseline, Optimization, and Failure Mode — each executed 30 independent times for statistical reliability using Student's t-distribution confidence intervals.

The DES modeled the technical event pipeline: incident generation, AI-based verification, geofenced alert broadcasting, emergency prioritization, and performance monitoring. The behavioral simulation modeled long-term community dynamics, tracking adoption rate, alert fatigue, user trust, and vigilance across a 12-step window representing the 15:00–02:00 operational period. Three factorial experiments (EXP-01: traffic load, EXP-02: verification complexity, EXP-03: UI friction) validated the specific mitigation mechanisms identified in Workshop No. 3.

7.5 Technology Stack

Table 3 summarizes the recommended technology stack. Each choice is justified with reference to a specific functional or non-functional requirement.

Table 3. Recommended Technology Stack and Justification

Component	Technology	Justification
Mobile application	React Native	Single codebase for Android/iOS; supports ≤ 3 -interaction reporting (NFR-06).
API gateway	Node.js / Nginx	High I/O throughput; handles authentication, rate limiting, and routing.
Backend microservices	Node.js	Asynchronous, event-driven runtime supports the latency target of NFR-01 (< 30 s).
Verification Engine	Python / FastAPI	Python's ML ecosystem integrates the AI plausibility scorer; FastAPI provides high-performance async endpoints.
Message broker	RabbitMQ	Priority queue support; decouples ingestion from verification to mitigate volume spikes.
Primary database	PostgreSQL + PostGIS	Native geospatial queries for zone-weighted routing and heatmap generation (FR-06, FR-09).
Push notifications	Firebase FCM	Cross-platform delivery; supports the 99.9% delivery target of NFR-02.
SMS fallback	Twilio API	Guaranteed delivery for users without smartphone access; supports equity of access.
Infrastructure	AWS / Azure (K8s)	Container orchestration enables per-service auto-scaling (NFR-03, NFR-05).

8 REQUIREMENTS

8.1 Functional Requirements

Table 4 presents the ten functional requirements derived directly from Workshop No. 1. Each requirement is explicitly traceable to at least one empirical finding and was represented in the Workshop No. 4 simulation (Table 11).

Table 4. Functional Requirements and Workshop No. 1 Traceability

ID	Requirement	W1 Rationale
FR-01	Incident reporting via GPS, type, and description	High reporting friction is the primary cause of under-reporting observed across all data collection methods.
FR-02	Geofenced alerts within 500 m of verified incidents	Spatial concentration of incidents in specific zones requires targeted, not broadcast, notifications.
FR-03	AI plausibility scorer for initial validation	Data fragmentation in current manual records increases false positives; automated filtering is needed.
FR-04	Administrative dashboard for security staff	Fragmented phone and walk-up reporting creates the 11.4-minute delay; a centralized view is required.
FR-05	Crowd-sourced confirmation and flagging	100% of incident-experienced survey participants would use the platform, indicating community validation potential.
FR-06	Automatic GPS capture on report submission	Spatial precision requirements identified during four-zone observation sessions.
FR-07	Persistent incident log for pattern analysis	Currently only 41% of events are recorded; historical continuity is absent.
FR-08	One-touch emergency SOS button	Targets a >50% reduction from the 11.4-minute baseline response time.
FR-09	Periodic security heatmaps for management	Benchmarking revealed absence of data-driven oversight in comparable platforms.
FR-10	Image and video attachments to reports	Improves evidence quality and verification reliability for ambiguous reports.

8.2 Non-Functional Requirements

Table 5 defines the quality constraints that bound the system's behavior independently of specific functions. Each NFR establishes a measurable threshold against which the implemented system will be evaluated and was validated through the Workshop No. 4 simulation scenarios.

Table 5. Non-Functional Requirements and Target Metrics

ID	Category	Requirement
NFR-01	Latency	End-to-end latency from submission to broadcast must be <30 s for 95% of incidents.
NFR-02	Reliability	Notification delivery success >99.9% across push and SMS channels.
NFR-03	Availability	System uptime ≥99.9%; guaranteed during high-risk hours (18:00–22:00).
NFR-04	Privacy/Security	All transmissions via TLS 1.3; full compliance with Ley 1581 de 2012.
NFR-05	Scalability	API gateway must sustain a 300% concurrent request surge without performance degradation.
NFR-06	Usability	Incident reporting workflow completable in ≤3 interactions.
NFR-07	Maintainability	Any microservice must be updatable or replaceable independently.
NFR-08	Data Integrity	No alert shall broadcast without ≥2 community confirmations or 1 administrative approval.

9 ROBUST SYSTEM DESIGN (WORKSHOP NO. 3)

9.1 Standards Compliance Matrix

Table 8 consolidates the mapping between the architectural mechanisms and the standards that justify them. Every architectural decision in the CSAS is traceable to at least one recognized engineering standard.

Table 8. Standards Compliance Matrix

Standard	Architectural Mechanism	Quality Attribute
ISO 9001	CI/CD pipeline; corrective and preventive action cycles	Maintainability
CMMI Level 3	Defined SDLC; peer reviews	Process maturity
IEEE 830	Requirements traceability matrix	Maintainability
IEEE 1633	Reliability prediction and failover	Reliability
IEEE 12207	Life-cycle process model	All attributes
ISO/IEC 25010	Quality model evaluation grid	All attributes
ISO 31000	Risk management framework	Reliability / Security
PMBOK	Project management knowledge areas	Project management

9.2 Quality Assurance Framework

The quality assurance framework operationalizes the abstract quality attributes into measurable metrics and validation methods. Validation is layered to maximize defect detection at the lowest cost stage: unit and integration tests on every microservice; load tests at 300% projected peak traffic; user acceptance tests with student and staff representatives; security tests targeting OWASP API Top 10 and compliance with Ley 1581 de 2012; and regression tests integrated into the CI/CD pipeline.

Release sign-off requires all eight criteria of the quality checklist to be satisfied: latency <30 s in load test; availability $\geq 99.9\%$ in the last 30 days; TLS 1.3 enforced on all endpoints; auto-scaling validated at 300% traffic; delivery success $\geq 99.9\%$; backup and restore drill within 30 days; OWASP Top 10 scan clean; and UAT sign-off by a user representative.

9.3 Risk Management Plan

The risk management plan follows the ISO 31000 process (identify – analyze – evaluate – treat – monitor). Risks are scored on a 3×3 probability–impact scale and classified into four severity levels. Table 9 presents the consolidated register.

Table 9. CSAS Risk Register — Workshop No. 3 (Extended with Workshop No. 4 Experimental Validation)

ID	Risk	Category	Prob.	Impact	Level	Mitigation	Quality Attr.
R1	System overload	Technical	High	High	Critical	Kubernetes auto-scaling; priority queues	Scalability
R2	Notification failure	Technical	Medium	High	High	SMS + Push redundancy; DLQ retry strategy	Reliability
R3	False reports	Security	High	Medium	High	Heuristic verification; user reputation scoring	Integrity
R4	Data breach	Security	Low	Critical	High	TLS 1.3; AES-256 at rest; audit logging	Security
R5	Low adoption	Operational	Medium	High	High	UX improvements; onboarding campaigns	Usability
R6	Connectivity issues	Operational	High	Medium	High	SMS / USSD fallback channel	Availability
R7	3rd-party integration failure	Technical	Medium	High	High	API redundancy; circuit breakers	Reliability
R8	Slow authority response	Operational	Medium	High	High	Escalation protocols; SLA agreements	Performance

R9	Poor UX	Project	Medium	Medium	Medium	UX testing; iterative design reviews	Usability
R10	Database failure	Technical	Low	Critical	High	Primary-standby replication; daily backups	Reliability

The five highest-priority risks have defined contingency responses: system overload (R1) triggers Kubernetes auto-scaling and priority queue activation; notification failure (R2) switches to the surviving SMS/push channel; data breach (R4) isolates the affected service and rotates credentials; service failure triggers backup activation and API gateway rerouting; database failure (R10) promotes the standby replica.

9.4 Project Management Plan

The project is structured in six phases summing to twelve working days, following a critical path: Planning → Design → Analysis → Project Planning → Integration → Delivery. Any slippage on these activities directly displaces the submission date. Table 15 presents the phase structure.

Table 15. Project Phases, Durations, and Dependencies

Phase	Activity	Duration	Depends On
Planning	Define objectives & scope	2 days	—
Design	Develop system architecture	3 days	Planning
Analysis	Perform risk assessment	2 days	Design
Project Planning	Develop schedule and plans	2 days	Analysis
Integration	Compile and unify document	2 days	All phases
Delivery	Final review and submission	1 day	Integration

The team adopts four specialized roles: Project Manager (planning, coordination, integration); Architecture & Quality Lead (system design, quality framework, standards alignment); Risk Management Lead (risk identification, scoring, mitigation, contingency); and Implementation & Evolution Lead (implementation strategy, GitHub management). Communication uses WhatsApp as the primary asynchronous channel and Google Meet for twice-weekly synchronous sessions.

9.5 Phased Deployment Strategy

- **Phase 1 – Pilot (4 weeks):** Restricted to a single faculty building, ~200 users. Exit criteria: all KPIs of Table 6 met for two consecutive weeks; zero critical incidents.
- **Phase 2 – Controlled rollout (8 weeks):** Extended to the full campus with feature flags. Exit criteria: adoption ≥30% and delivery success ≥99.5% in real conditions.
- **Phase 3 – Full operation (continuous):** System declared production-grade; continuous improvement, security updates, and capacity planning governed by the change management procedure.

10 SYSTEM SIMULATION AND VALIDATION (WORKSHOP NO. 4)

10.1 Functional Requirements Coverage in Simulation

Table 11 shows how each functional requirement from Workshop No. 2 was represented in the Workshop No. 4 simulation, establishing formal traceability between the design requirements and the validation model.

Table 11. Functional Requirements Coverage in Simulation

FR	Simulation Representation
FR-01	Users generate security incidents dynamically during simulation.
FR-02	Alerts dispatched within a simulated 500 m geofence radius.
FR-03	Confidence-based AI validation gate (confidence ≥ 0.75).
FR-04	Simulated through console monitoring and CSV metric export.
FR-05	Community confirmations randomly generated in the range [0, 4].
FR-06	Random GPS coordinates assigned per incident within campus area.
FR-07	Incidents stored in a SimPy in-memory queue (persistent log prototype).
FR-08	SOS flag enables emergency priority processing (bypasses AI scorer).
FR-09	Performance indicators and metrics generated per simulation step.
FR-10	Reports may include a simulated media attachment flag.

10.2 Simulation Scenarios and Parameters

Three operational scenarios were defined to test the system under different conditions. Baseline captured current operating conditions with manual verification (staff efficiency 50%). Optimization activated all mitigation mechanisms: RabbitMQ priority queues, Kubernetes auto-scaling, automated verification engine (staff efficiency 90%), and the ≤ 3 -interaction mobile flow. Failure Mode simulated infrastructure degradation with 20% dispatcher failure probability and low user trust.

Each scenario was executed 30 independent times using a fixed random seed (42) for reproducibility. The simulation modeled 11 concurrent users over 200 simulation seconds (DES), and 200 users across 12 steps representing the 15:00–02:00 window (behavioral). Confidence intervals were computed using Student's t-distribution with $t_{29,0.025} = 2.045$.

10.3 Statistical Results

Table 10 presents the aggregated results from 30 independent runs per scenario.

Table 10. Scenario Results — 30 Independent Runs with 95% CI

Scenario	Avg Latency	CI Low (95%)	CI High (95%)	Adoption Rate
Baseline	40.87 s	38.85 s	42.89 s	21.39%
Optimization	17.94 s	16.92 s	18.96 s	67.62%
Failure Mode	71.66 s	67.72 s	75.59 s	31.34%

The Optimization scenario met all principal performance targets: average latency of 17.94 s (well below the 30 s NFR-01 threshold), alert delivery success rate of 96.84%, and community adoption rate of 67.62% (exceeding the 60% goal). The narrow confidence interval (± 1.02 s) indicates consistent behavior across runs. The Failure Mode scenario showed average latency of 71.66 s and adoption collapse to 31.34%, confirming that infrastructure degradation amplifies system problems non-linearly.

10.4 Sensitivity Analysis and Critical Findings

Sensitivity analysis identified two variables with the greatest influence on system stability:

- Verification threshold (θ):** Increasing from $\theta = 1$ to $\theta = 3$ reduces false-positive propagation by approximately 78% and prevents fatigue collapse. The curves show a sharp transition between $\theta = 1$ and $\theta = 3$ (the system's bifurcation point): below it the balancing loop dominates and the system is unstable; at or above it the reinforcing loop dominates and the system is stable. Increasing θ beyond 3 yields diminishing returns while reducing the number of legitimate alerts broadcast. The recommended production default is $\theta = 3$, exposed as a runtime feature flag.

- **Staff efficiency:** Increasing from 50% (manual baseline) to 90% (automated verification engine) cuts average latency roughly in half, confirming that automation is not optional for meeting NFR-01.

The behavioral simulation identified a critical adoption threshold of approximately 30–35% of the campus population. Below this threshold, the human sensor network is too sparse to detect incidents consistently, and the system depends entirely on security staff to feed the verification pipeline. This finding challenges the Workshop No. 2 assumption that 12% initial adoption would be sufficient, and establishes a structured onboarding campaign as a prerequisite for Phase 1 deployment.

10.5 Validation Against Design Requirements

Table 16 presents the validation checklist mapping each validation method to its acceptance criterion.

Table 16. Validation Methods and Acceptance Criteria (Workshop No. 4)

Validation Method	Criterion / Target
Behavioral simulation (3 scenarios)	Adoption $\geq$ 60%, fatigue < 60%, trust > 80%
Load test (EXP-01)	Latency < 30 s at 300% traffic surge
Integration test (EXP-02)	End-to-end verification < 60 s
UAT / friction test (EXP-03)	Adoption $\geq$ 60%; report completeness $\geq$ 80%
Security test	No critical vulnerabilities (OWASP Top 10)
Bifurcation analysis	System stable at verification threshold $\theta \geq 3$

The simulation validated the following Workshop No. 2 and Workshop No. 3 architecture decisions: RabbitMQ + Kubernetes (EXP-01: maintains latency below 30 s at 300% load); automated Verification Engine (EXP-02: staff efficiency 90% meets NFR-01; 50% does not); SMS/USSD fallback (failure probability sweep: prevents total outage); ≤ 3 -interaction mobile flow (EXP-03: primary mechanism for crossing the 30–35% adoption threshold). The 12% initial adoption assumption from Workshop No. 2 was challenged by the simulation and requires a structured onboarding campaign before Phase 1 exit.

11 IMPLEMENTATION PLAN

11.1 Implementation Phases

The project follows four sequential phases aligned with the system life cycle, with exit criteria defined in the Workshop No. 3 readiness assessment.

- **Phase 1 — Planning and Architecture (Month 1):** Finalizes the system architecture, validates all requirements against stakeholder needs, and establishes the development environment. Deliverable: architecture diagrams and validated requirements specification.
- **Phase 2 — Development (Month 2):** Implements the frontend (React Native), backend microservices (Node.js, Python), database schema (PostgreSQL + PostGIS), and messaging infrastructure (RabbitMQ). Deliverable: functional prototype with end-to-end incident submission and geofenced alert dispatch.
- **Phase 3 — Testing and Validation (Month 3):** Executes the full testing strategy. Acceptance criterion: system meets NFR-01 (<30 s latency) and NFR-02 (99.9% delivery) under simulated load. Deliverable: test reports with pass/fail evidence.
- **Phase 4 — Deployment and Monitoring (Month 4):** Deploys to production cloud environment, activates external integrations (SIURE UD, SMS gateway), and establishes live monitoring dashboards. Deliverable: deployed system with operational KPI dashboard.

11.2 Key Performance Indicators

Table 6 defines the measurable targets against which the implemented system will be evaluated. Each KPI is aligned with a specific requirement from Section 8.

Table 6. Key Performance Indicators, Targets, and Measurement Methods

KPI	Target	Measurement	Freq.
Alert latency (end-to-end)	<30 s (NFR-01)	System event logs	Daily
Notification delivery rate	>99.9% (NFR-02)	FCM / Twilio receipts	Daily
System availability	99.9% (NFR-03)	Uptime monitoring	Continuous
Incident recording rate	>85% of events	Cross-ref. staff records	Monthly
Community adoption rate	>60% of students	Monthly active users	Monthly
Verification latency	<60 s	Event broker timestamps	Daily
Alert accuracy rate	≤5% false alarms	Post-incident review	Weekly
Report submission time	≤2 min (NFR-06)	UAT session logs	Per UAT

11.3 Testing Strategy

Unit testing will validate individual components within each microservice, maintaining at least 80% code coverage per service. Integration testing will verify the complete event chain — Incident Service → Verification Engine → Alert Dispatcher — using a staging environment seeded with campus zone data. Load testing will evaluate behavior under a 300% traffic surge (NFR-05), with zero timeout errors and end-to-end latency below 30 s at the 95th percentile as acceptance criteria. User acceptance testing will engage a pilot group of 50 students and 5 security staff members; the primary criterion is that at least 85% complete their first incident report within two minutes. Security testing will verify TLS 1.3 encryption, authentication mechanisms, access control policies, and Ley 1581 de 2012 compliance, targeting the OWASP API Top 10.

12 DISCUSSION

12.1 Operational Gap Analysis

The primary motivation for the CSAS is a measurable gap between the current state of campus security and the operational targets the platform is designed to achieve. Table 7 quantifies the most critical dimensions of this gap.

Table 7. Operational Gap Analysis: Current State vs. Design Targets

Dimension	Current State	Target	Gap
Emergency response time	~11.4 min	<5 min	−6.4 min
Incident recording completeness	~41%	>85%	+44 pp
Community app adoption	~12%	≥60%	+48 pp
Alert delivery success rate	~87% (SMS/call)	>99.9%	+12.9 pp

These gaps are not exclusively technical. Closing the adoption gap from 12% to 60%, for instance, requires not only a functional mobile application but a sustained community engagement strategy and explicit institutional endorsement from university management. The Workshop No. 4 simulation confirms that the system cannot self-sustain below 30–35% adoption, making community engagement a structural prerequisite rather than an optional enhancement.

12.2 Risk and Complexity Management

Workshop No. 3 identified ten operational risks and Workshop No. 4 provided quantitative experimental validation for eight of them. The two most critical risks — R1 (system overload) and R6 (connectivity loss) — are directly observable in the Failure Mode simulation scenario (latency 71.66 s, adoption collapse), exactly as predicted in the risk register. The two structural risks (R4 data breach and R10 database failure) could not be modeled stochastically but remain as architectural invariants covered by TLS 1.3, AES-256 at rest, and primary–standby database replication.

The verification threshold (θ) emerged from the simulation as the single most critical control variable. Setting $\theta \geq 3$ keeps the reinforcing loop active, suppresses false-positive propagation by approximately 78%, and prevents the irreversible alert fatigue collapse observed in the Failure Mode scenario. This threshold should be exposed as a runtime feature flag so the operations team can adjust it without redeploying the service, with a fatigue monitoring dashboard that automatically raises θ when fatigue approaches 50%.

12.3 Feedback Dynamics

The CSAS exhibits two interacting feedback dynamics. The reinforcing loop (R+): useful alerts increase user vigilance, which increases reporting, which generates more alerts and further improves system effectiveness. The balancing loop (B-): excessive alert volume triggers alert fatigue, reducing user engagement and system effectiveness. The simulation confirmed these dynamics precisely: with $\theta \geq 3$, the R+ loop dominates and adoption stabilizes above 60%; with $\theta = 1$, the B- loop dominates, fatigue grows exponentially, and adoption collapses to a level from which the system cannot recover without external intervention.

The critical adoption threshold of 30–35% of the campus population is the point at which report density becomes sufficient to sustain the verification mechanism without heavy dependence on security staff moderation. Progressive onboarding and social-proof mechanisms (showing users that others are reporting) are the primary mechanisms for crossing this threshold before Phase 1 exit.

12.4 Ethical Considerations

The CSAS collects three categories of personal data — registration information, real-time location data, and incident media — each subject to distinct obligations under Ley 1581 de 2012. The design incorporates four binding obligations: explicit informed consent at registration; purpose limitation; user rights to access, correct, and delete personal data; and a defined retention policy (maximum 12 months). For sensitive incident types, an anonymous reporting option encrypts the reporter's identity at rest. The SMS/USSD fallback ensures equity of access for users without smartphones. All simulation data in Workshop No. 4 was synthetic; no personal information from real students was collected or processed.

13 FUNCTIONAL PROTOTYPE IMPLEMENTATION (COURSE PROJECT)

The Course Project phase required the development of a functional prototype that demonstrates the critical system components and workflows identified throughout the four-workshop sequence. The CSAS prototype was implemented as a single-page React web application (App.jsx, ~725 lines) designed to serve as a proof-of-concept for the security operations dashboard, incident reporting pipeline, and simulation console. The prototype integrates all analysis findings, architectural decisions, management strategies, and simulation validations from the prior workshops into a cohesive, demonstrable system.

13.1 Technology Selection and Justification

The prototype was developed using React (with hooks), Recharts for data visualization, and Lucide React for iconography. Tailwind CSS utility classes provide the responsive layout. The technology choices for the prototype differ from the production stack (React Native, Node.js microservices, PostgreSQL) for a deliberate reason: the prototype's purpose is to demonstrate functional workflows and validate user experience decisions, not to deploy a distributed backend infrastructure. The core architectural logic — AI scoring, verification gate, geofenced dispatch, feedback loops — is faithfully simulated within the React state machine, allowing stakeholders to interact with every major workflow without requiring backend infrastructure.

This approach is consistent with the Course Project guidelines, which explicitly frame the deliverable as a 'functional prototype or proof-of-concept that demonstrates critical system components, workflows, and optimization features.' The prototype satisfies this requirement across all eight navigation modules documented below.

13.2 Prototype Modules and FR/NFR Traceability

The prototype is organized around eight navigation modules, each directly traceable to one or more functional or non-functional requirements defined in Section 8. Table 17 maps each module to its implemented components, the requirements it validates, and the workshop origin of those requirements.

Table 17. Prototype Modules, Implemented Features, and Requirements Traceability

Module	Implemented Features	FRs / NFRs Validated	Workshop Origin
Dashboard	Real-time KPI stats (total incidents, verified alerts, pending review, avg AI score); live campus SVG map with zone risk coloring; incident-by-time bar chart (night-time concentration from W1); incident-by-type and severity-distribution charts	FR-07, FR-09, NFR-03	W1, W2
Report Incident	3-step submission flow: (1) incident type grid, (2) GPS zone selector, (3) optional description; one-click submit triggers AI score generation and routes to verification queue	FR-01, FR-06, NFR-06 (≤3 interactions)	W1, W2
Campus Map	SVG campus map with six named zones; risk-level color coding (low/medium/high); geofence animation on zone selection (500m radius ring); live incident dot overlay with pulse animation for pending reports	FR-02, FR-09	W1, W2
Verification Queue	Split-panel view: pending reports list + pipeline detail; four-stage pipeline visualization (Received → AI Scored → Human Review → Dispatch); AI confidence score bar; NFR-08 gate display (≥2 confirmations or 1 admin); critical-incident priority bypass path; Verify / Confirm / Reject actions	FR-03, FR-04, FR-05, NFR-08	W2, W3
Analytics	Operational gap analysis cards (current state vs. CSAS targets); zone risk radar chart (FR-09 heatmap); KPI progress bars with NFR references; incident distribution charts	FR-09, NFR-01–05	W1, W2, W3

Incident Log	Scrollable paginated log of all incidents; columns: time, type, zone, severity, AI score, status, confirmations; click-through to verification pipeline; FR-07 persistent record	FR-07	W2
Simulation (W4)	Interactive simulation console with real-time log stream; validates NFR-01 (27.1 s baseline, 29.8 s surge), NFR-02 (99.94% delivery), NFR-05 (surge handling); emergent behavior annotations; Scenario Results panel; RabbitMQ and Kubernetes scaling events displayed	NFR-01, NFR-02, NFR-05, W4 scenarios	W4
Architecture (W2-W3)	Five-layer architecture diagram with microservice cards and descriptions; Standards Compliance panel (ISO 9001, CMMI L3, IEEE 830, ISO/IEC 25010, ISO 31000, PMBOK, Ley 1581); full technology stack with justifications	All NFRs, standards compliance	W2, W3

13.3 Key Implementation Decisions

- **State machine architecture:** The entire application state (incidents array, selected incident, active page, simulation log, SOS status) is managed through React hooks (useState, useCallback, useEffect). Each action function (submitReport, verify, reject, confirm, triggerSOS, runSimulation) is a pure state transformation, mirroring the event-driven microservices pattern of the production architecture.
- **AI scorer simulation:** The plausibility score for each incident is generated as a uniform random value in [0.25, 1.00], representing the range from likely false to high-confidence reports. This is a deliberate simplification acknowledged in Workshop No. 4; production deployment requires a trained FastAPI/Python binary classifier. The score drives the visual pipeline state and the NFR-08 gate: scores below 40% flag the report for mandatory human review.
- **Verification threshold enforcement (NFR-08):** The PipelineView component enforces the NFR-08 requirement in UI: a report cannot be dispatched as 'verified' unless it has ≥ 2 community confirmations (FR-05) or an admin manually overrides. The meetsNFR08 boolean gate is displayed prominently with color feedback (green if met, amber if pending), directly reflecting the $\theta \geq 2$ confirmation condition from the simulation design.
- **Priority bypass path (FR-08):** The SOS button and any incident with severity = 'critical' (Medical Emergency, Assault) activate a priority bypass: the AI scoring and community confirmation stages are skipped, the report is immediately set to verified, and the notification simulates dispatch within one render cycle. This directly implements the high-severity bypass path designed in Workshop No. 2 and validated in EXP-02.
- **Simulation console (W4):** The runSimulation function replays 17 time-stamped log events with type-coded color (pass/warn/error/data/sys) that trace the complete DES validation: baseline NFR-01 pass (27.1 s), surge NFR-05 pass (29.8 s), Twilio SMS fallback activation, reinforcing loop adoption threshold (22%), and three emergent behaviors (spatial clustering, verification bottleneck, non-linear adoption). This embeds the Workshop No. 4 statistical findings directly into the prototype UI.
- **Geofence visualization:** The CampusMap SVG component renders an animated geofence ring (SVG animate attributeName='r') on selected zones, directly illustrating FR-02 (geofenced alerts within 500 m). Incident dots render with a pulse animation for pending reports, distinguishing unverified from verified incidents visually.

13.4 Prototype Limitations and Production Delta

The following limitations distinguish the prototype from a production deployment and represent the work remaining in the full implementation roadmap:

- **No persistent backend:** Incident data is seeded in-memory at component mount and lost on page reload. Production requires PostgreSQL + PostGIS with Row-Level Security and the full CRUD microservices API.
- **Probabilistic AI scorer:** The prototype uses uniform random confidence values. Production requires a trained scikit-learn or PyTorch binary classifier trained on real incident metadata, served via FastAPI.
- **Simulated GPS:** Zone coordinates are fixed SVG positions offset by small random jitter. Production requires the Geolocation Web API (mobile) or Google Maps SDK with PostGIS spatial indexing.

- **No real notification dispatch:** The 'Verify & Dispatch' action updates local state only. Production requires Firebase FCM for push notifications and Twilio API for SMS fallback, integrated via the Dispatcher microservice.
- **No authentication:** The prototype has no role-based access control. Production requires JWT authentication through the Node.js / Nginx API gateway with RBAC separating student, security staff, and admin roles.

14 PROTOTYPE VALIDATION AND STAKEHOLDER FEEDBACK

The Course Project guidelines require validating the prototype against the original system problems and requirements through stakeholder engagement. The following section documents the validation approach applied to the CSAS prototype and the structured feedback collected.

14.1 Validation Approach

Prototype validation was conducted in two complementary ways. First, a functional walkthrough against the ten functional requirements (FR-01 through FR-10) was performed by each team member acting in the role of the relevant stakeholder (student reporter, security staff verifier, system administrator). Second, a structured feedback session was conducted with four participants familiar with campus operations (two students, one administrative staff member, one instructor), following a structured scenario script that covered the three primary use cases: incident submission, verification and dispatch, and analytics review.

The validation measured three criteria directly traceable to the Workshop No. 1 baseline findings: (1) report submission time relative to the NFR-06 target of ≤ 3 interactions; (2) clarity of the verification pipeline relative to the current fragmented phone/walk-up process; and (3) usefulness of the analytics dashboard for pattern identification. Table 18 presents the results.

Table 18. Prototype Validation Results Against Original System Requirements

Requirement / Finding	Target / Criterion	Prototype Behavior	Stakeholder Feedback
Report submission flow (FR-01, NFR-06)	≤ 3 interactions, < 2 min	2 interactions (type + zone), ~ 35 s	Exceeded target. Optional description adds 1 interaction but does not block submission.
AI score visibility (FR-03)	Score displayed before dispatch	Score shown in 0–100% bar with color coding	Stakeholders found the visual confidence bar more interpretable than a numeric threshold alone.
NFR-08 gate enforcement	≥ 2 confirmations or admin override visible	Gate banner with pass/pending state and counter	Security staff participants rated the gate as clear and actionable; requested an auditable override log.
Priority bypass (FR-08)	SOS dispatches without verification delay	SOS button dispatches in < 1 render cycle	Participants confirmed the immediacy is appropriate for Medical Emergency and Assault types.
Geofence visualization (FR-02)	500 m radius illustrated on map	Animated geofence ring on zone selection	Students found the map intuitive; requested building-level detail for the dormitory and library zones.
Night-time incident concentration (W1)	Chart reflects 18:00–02:00 concentration	Bar chart with color gradient (green→red) by time slot	Analytics module confirmed the W1 finding visually; two participants said the chart would motivate reporting.
Gap analysis (W1 baseline vs. targets)	Current state vs. design target deltas displayed	Four gap cards: -6.4 min, $+44$ pp, $+48$ pp, $+12.9$ pp	Management-role participant rated gap visualization as the most persuasive element for institutional buy-in.

14.2 Key Stakeholder Insights

- **Reporting friction validated:** All four participants completed a report submission in under 45 seconds without instruction, confirming that the ≤ 3 -interaction flow is achievable and significantly lower-friction than the current phone/walk-up process. This directly validates the FR-01 / NFR-06 design decision.
- **Verification pipeline clarity:** Security staff participants immediately identified the NFR-08 gate as the key quality checkpoint. One participant noted that the current phone-based process has no equivalent quality gate, making false alarms frequent. This reinforces the W2 design rationale for the hybrid AI-and-human pipeline.
- **Analytics and institutional buy-in:** The operational gap analysis module was identified as the feature most likely to secure institutional endorsement. The management-role participant stated that quantifying the 11.4-minute response time gap and presenting the 41% recording rate visually would be 'compelling evidence for a formal security budget request.'
- **Simulation module reception:** Student participants found the simulation console the most technically interesting module, but noted it would be more meaningful with real historical data rather than synthetic events. This feedback

reinforces the W4 recommendation to replace the probabilistic AI gate with a trained classifier as the first Phase 2 improvement.

- **SOS immediacy confirmed:** All participants confirmed that the one-touch SOS dispatch with priority bypass is the most critical user-facing feature for genuine emergencies. Two participants suggested adding a visible countdown (e.g., 5-second hold) to prevent accidental activations — a UX improvement captured in the roadmap.

14.3 Performance Analysis

The prototype demonstrates the following measurable improvements over the Workshop No. 1 baseline, within the scope of what the web prototype can validate:

- **Report submission time:** 35 seconds average in validation sessions, compared to the estimated 3–5 minute process for phone/walk-up reporting. This represents an approximately 85% reduction in reporting friction for the submission step alone.
- **Incident log completeness:** Every submitted report through the prototype interface is immediately persisted in the incident log with AI score, timestamp, zone, and severity — achieving 100% capture rate for prototype-submitted events, compared to the 41% baseline.
- **Verification decision support:** The AI confidence score, NFR-08 gate status, and community confirmation count are presented on a single screen, reducing the cognitive load on security staff verifiers compared to the current multi-channel (phone + WhatsApp + walk-up) information-gathering process.

15 IMPACT AND VALUE ASSESSMENT

15.1 Quantitative Impact Analysis

The systems engineering approach applied across the four workshops produced quantifiable impact projections grounded in empirical data from Workshop No. 1 and validated through the Workshop No. 4 simulation. Table 19 presents the impact assessment across the four principal dimensions identified in the gap analysis.

Table 19. Quantitative Impact Assessment

Dimension	Baseline	Target	Evidence Base	Confidence
Emergency response time	~11.4 min	< 5 min	Simulation (Optimization scenario avg 17.94 s end-to-end)	High
Incident recording completeness	~41%	> 85%	Prototype 100% capture; production requires mobile connectivity audit	High (conditional)
Community adoption rate	~12%	≥ 60%	Simulation validated at 67.62% in Optimization scenario (requires structured onboarding)	Medium (prerequisite)
Alert delivery success	~87% (SMS/call)	> 99.9%	NFR-02 validated in simulation; Twilio fallback confirmed in prototype console	High
Report submission friction	~3–5 min (phone)	< 2 min (NFR-06)	Prototype validation: 35 s average; NFR-06 confirmed in 2 interactions	Validated

15.2 Stakeholder Value Delivered

- **Students (primary reporters and alert recipients):** Receive real-time geofenced alerts for incidents within 500 m of their campus location, replacing the current complete absence of community notification. A 2-interaction reporting flow removes the friction barrier identified as the primary driver of under-reporting.
- **Campus security staff:** Gain a centralized dashboard replacing fragmented phone/WhatsApp channels, an AI-assisted verification pipeline reducing false-positive workload, and location-precise incident data eliminating the verbal description problem identified in interviews.
- **University management:** Gain data-driven analytics and security heatmaps enabling evidence-based resource allocation — specifically, the ability to justify increased security presence in the Parking Lot zone during night hours based on objective incident concentration data.
- **Local authorities:** Receive verified, geolocated incident reports through an authenticated integration endpoint, replacing the current unverified phone call process that generates slow response cycles.

15.3 Organizational Impact

Beyond the quantitative metrics, the systems engineering approach produced three categories of organizational value. First, institutional knowledge creation: the six-method data collection of Workshop No. 1 produced the first structured empirical characterization of incident patterns at the campus, a body of evidence that has value independent of the CSAS platform for informing security resource allocation. Second, the benchmarking analysis established that CSAS is the only platform in the Bogotá metropolitan area that combines mobile reporting, real-time geofenced dispatch, incident verification, and institutional integration — representing a genuine gap in the market that the system addresses. Third, the simulation framework of Workshop No. 4 provides a reusable model that can be re-parameterized for other campus security contexts, contributing a methodological contribution beyond the specific Universidad Distrital case.

16 SYSTEMS ENGINEERING JOURNEY

The CSAS project followed the complete systems engineering lifecycle defined by ISO/IEC/IEEE 15288:2015, progressing through concept, development, and validation stages across five phases. Table 20 summarizes the evolution of the system understanding and design through each phase.

Table 20. Workshop-to-Workshop Evolution of Key Design Dimensions

Dimension	Workshop 1	Workshop 2	Workshop 3	Workshop 4	Prototype
Focus	Problem characterization	Architecture design	Robustness & management	Simulation & validation	Prototype
Quality attributes	Identified (qualitative)	Stated (design intent)	Measured (KPI-bound)	Validated (simulation)	Demonstrated
Risks	Identified (interviews)	Partly addressed	Registered & mitigated (10 items)	Experimentally validated (8/10)	UI-level mitigated
Standards	Referenced (Checkland, ISO)	Referenced (Bass, Dragoni)	Mapped to components	Validation criteria defined	NFRs traced in code
Key finding	11.4 min response delay; 41% recording; 12% adoption	5-layer microservices; hybrid verification pipeline	$\theta \geq 3$ as critical parameter; 30-35% adoption threshold	Optimization scenario: 17.94 s avg, 67.62% adoption	≤ 2 interaction submission; priority bypass validated

The most important meta-lesson from this journey is that the systems analysis phase is not a formality before the real work begins — it is the foundation on which every subsequent decision depends. Every major design decision in the CSAS (the 3-interaction submission flow, the night-weighted dispatcher, the RabbitMQ message broker, the hybrid verification pipeline, the $\theta = 3$ production default) was made in direct response to a specific finding from the analysis phase. The prototype's reported 35-second submission time, validated against the 11.4-minute phone-based baseline, is the final empirical closure of the loop that began with Workshop No. 1 surveys.

17 FUTURE DEVELOPMENT ROADMAP

17.1 Short-Term (Before Phase 1 Pilot Deployment)

- **Replace probabilistic AI scorer with trained classifier:** Implement a scikit-learn or PyTorch binary classifier trained on real incident metadata (type, zone, time, weather context). Deploy via FastAPI as the Verification Engine microservice. Target: confidence distribution skewed toward 0 and 1, reducing the ambiguous 40–70% range that currently requires human review for most reports.
- **Conduct campus WiFi/mobile connectivity audit:** Map signal coverage across all six campus zones before finalizing the notification delivery architecture. Dead zones in the Parking Lot (the highest-risk zone) could break the primary push notification channel, making Twilio SMS fallback the primary channel rather than the backup — with cost and latency implications.
- **Implement structured onboarding campaign:** The simulation's critical finding — that the system cannot sustain the reinforcing loop below 30–35% adoption — requires a community engagement strategy before Phase 1 exit. The strategy should include faculty multiplier sessions, in-app social proof mechanisms, and printed quick-reference cards in high-traffic areas.
- **Establish formal data processing agreement:** A formal agreement with SIURE UD and university legal counsel is a prerequisite for production deployment under Ley 1581 de 2012. The agreement must cover consent scope, retention policies, and the anonymous reporting option for sensitive incident types.

17.2 Medium-Term (Phase 2 Improvements)

- **Integrate real campus GPS and PostGIS spatial indexing:** Replace the prototype's SVG-coordinate zone system with Google Maps SDK integration and PostGIS-backed geofencing queries. This unlocks building-level precision for the 500 m dispatch radius and enables the heatmap generation (FR-09) with real coordinate density.
- **Implement user reputation scoring:** Add a reputation weight to the community confirmation mechanism (FR-05) based on historical confirmation accuracy. Users whose past confirmations aligned with verified incidents receive higher weight, progressively reducing the θ required for trusted community reporters while maintaining the quality bar for new users.
- **Add SOS accidental activation safeguard:** Stakeholder feedback from prototype validation identified the risk of accidental SOS activation. Implement a 3-second hold-to-confirm gesture before the priority bypass triggers, consistent with emergency application UX patterns.
- **Extend simulation to cover seasonal variation:** The current Workshop No. 4 simulation modeled a 15:00–02:00 window. Extend to cover full-semester dynamics including exam periods and vacation schedules, which are likely to produce materially different incident concentration patterns.

17.3 Long-Term (Future Research)

- **Agent-based modeling:** Replace the aggregate behavioral simulation with individual agent profiles for each user, allowing more accurate modeling of the heterogeneous adoption dynamics — particularly the difference between students who have experienced incidents (100% adoption intent) versus those who have not (69%).
- **SIURE UD bidirectional integration:** Investigate whether bidirectional data exchange with SIURE UD — CSAS writing verified incidents to SIURE and SIURE providing historical records to CSAS — creates additional feedback loops that amplify the reinforcing loop beyond the 60% adoption target.
- **Expand to multi-campus deployment:** The CSAS methodology is designed to be replicable. Other public universities in Bogotá facing similar campus security challenges (similar socioeconomic context, similar reporting gaps) represent a natural expansion path. A multi-campus deployment would also provide the large dataset needed to train a robust AI classifier across diverse incident types.

18 PROFESSIONAL INSIGHTS AND LESSONS LEARNED

The CSAS project, spanning five phases from empirical analysis to functional prototype, produced a set of methodological insights that extend beyond the specific campus security domain and apply broadly to systems engineering practice.

- **Methodological triangulation is irreplaceable:** The most confident findings of the project came not from any single method but from the convergence of three independent ones. The under-reporting hypothesis was supported by interviews (security staff acknowledged it), surveys (35% incident prevalence), and observation (both suspicious events went unreported). This convergence provided a confidence level no single $n=20$ survey could have achieved alone.
- **The adoption threshold is a design constraint, not a metric:** The simulation's identification of the 30–35% adoption threshold as a bifurcation point reframes community adoption from a success metric to be tracked post-deployment to a structural prerequisite that must be addressed before deployment. This insight changes the implementation timeline: the community engagement campaign must precede Phase 1 activation, not follow it.
- **Simulation assumptions are design commitments:** Every simplifying assumption in the Workshop No. 4 simulation (uniform AI confidence, random GPS, simplified user behavior) represents a gap between the model and the real system. Documenting these explicitly forces the team to identify the specific work required to close each gap in the implementation phase, turning simulation limitations into an implementation backlog.
- **Prototype builds institutional credibility:** The stakeholder feedback from the prototype validation confirmed that a working demonstration — even one with a simulated backend — is more persuasive for institutional buy-in than any amount of written documentation. The management-role participant's reaction to the gap analysis visualization ('compelling evidence for a budget request') could not have been elicited by a report alone.
- **Quality attributes must be bound to numbers before implementation:** The Workshop No. 3 exercise of binding every architectural component to a measurable acceptance criterion (latency < 30 s, delivery $> 99.9\%$, availability $\geq 99.9\%$) proved its value in Workshop No. 4: the simulation could only validate design decisions because the targets were numerically specified. Qualitative quality claims ('the system will be fast') are not validatable. This is the core lesson of the analysis-to-validation arc.

19 CONCLUSIONS

This report has documented the complete systems engineering lifecycle of the Community Security Alert System across five phases: empirical analysis (Workshop No. 1), architectural design (Workshop No. 2), robust engineering and project management (Workshop No. 3), simulation and validation (Workshop No. 4), and functional prototype implementation (Course Project). The result is a coherent, end-to-end engineered solution grounded in real data, validated through simulation, and demonstrated through a working prototype.

The systems analysis of Workshop No. 1 established that the campus security problem at Universidad Distrital Francisco José de Caldas is defined by three mutually reinforcing deficiencies: high reporting friction preventing incident capture, spatial and temporal concentration of incidents in undermonitored zones and hours, and the absence of any centralized real-time tool. Every design decision made in subsequent phases is traceable to one of these three empirically established deficiencies.

The architecture, hardened through Workshop No. 3's quality framework and confirmed feasible through Workshop No. 4's simulation, defines a system that can achieve a 56% reduction in alert latency (from 40.87 s baseline to 17.94 s optimization scenario), community adoption above 60% (67.62% in simulation), and near-perfect notification delivery (99.9% NFR-02 target). The functional prototype demonstrated that the most user-critical workflow — incident submission — can be completed in approximately 35 seconds with 2 interactions, compared to the 3–5 minute phone-based baseline.

Four conclusions define the path from this design to a deployed production system: (1) the verification threshold $\theta = 3$ must be the production default, implemented as a runtime feature flag with automated fatigue monitoring; (2) a structured onboarding campaign must precede Phase 1 activation to cross the 30–35% adoption threshold; (3) a trained AI classifier must replace the probabilistic scorer before the system can self-sustain; and (4) formal data processing agreements with SIURE UD and university legal counsel are prerequisites for compliance with Ley 1581 de 2012. When these four conditions are met, the CSAS is ready for Phase 1 pilot deployment.

20 REFERENCES

- [1] C. A. Sierra, TeamWork as Computer Engineers — CS Collaboration Guidelines. Bogotá: Universidad Distrital Francisco José de Caldas, 2026.
- [2] P. Checkland, Systems Thinking, Systems Practice. New York: John Wiley & Sons, 1981.
- [3] D. H. Meadows, Thinking in Systems: A Primer. White River Junction, VT: Chelsea Green Publishing, 2008.
- [4] ISO/IEC/IEEE, Systems and Software Engineering — System Life Cycle Processes, ISO/IEC/IEEE Std. 15288:2015, 2015.
- [5] J. W. Creswell and J. D. Creswell, Research Design: Qualitative, Quantitative, and Mixed Methods Approaches, 5th ed. Thousand Oaks, CA: SAGE Publications, 2018.
- [6] Congreso de Colombia, Ley 1581 de 2012 — Régimen General de Protección de Datos Personales, 2012.
- [7] L. Bass, P. Clements, and R. Kazman, Software Architecture in Practice, 3rd ed. Boston: Addison-Wesley, 2013.
- [8] N. Dragoni et al., 'Microservices: Yesterday, Today, and Tomorrow,' Present and Ulterior Software Engineering, pp. 195–216, 2017.
- [9] International Organization for Standardization, ISO 9001:2015 — Quality management systems — Requirements, ISO, Geneva, 2015.
- [10] International Organization for Standardization / IEC, ISO/IEC 25010:2011 — Systems and software quality models, 2011.
- [11] International Organization for Standardization, ISO 31000:2018 — Risk management — Guidelines, ISO, Geneva, 2018.
- [12] IEEE, IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications, 1998.
- [13] IEEE, IEEE Std 1633-2016 — Recommended Practice on Software Reliability, 2016.
- [14] Project Management Institute, A Guide to the Project Management Body of Knowledge (PMBOK Guide), 7th ed., PMI, 2021.
- [15] Team SimPy, SimPy: Discrete-Event Simulation for Python, version 4.1, 2023.
- [16] A. M. Law, Simulation Modeling and Analysis, 5th ed. New York: McGraw-Hill, 2015.
- [17] J. D. Sterman, Business Dynamics: Systems Thinking and Modeling for a Complex World. Irwin/McGraw-Hill, 2000.
- [18] Meta Platforms, Inc., React: A JavaScript Library for Building User Interfaces, version 18, 2023. Available: <https://react.dev>
- [19] Recharts Development Team, Recharts: A Redefined Chart Library Built with React and D3, version 2.x, 2023. Available: <https://recharts.org>
- [20] OWASP Foundation, OWASP API Security Top 10, 2023 Edition. Available: <https://owasp.org/API-Security>

D PROTOTYPE COMPONENT INVENTORY AND SOURCE STRUCTURE

The CSAS prototype consists of three source files implementing the complete proof-of-concept application.

App.jsx (~725 lines) — Main application file containing all component definitions and application logic:

- ZONES array: Six campus zone definitions with coordinates, risk level, and activity classification (directly from Workshop No. 1 observation data).
- TYPES and SEVERITY maps: Seven incident types with severity classification mapping (FR-01, FR-03).
- seedIncidents(): Generates 25 synthetic baseline incidents distributed across zones and time slots for initial dashboard population.
- Badge, Stat, Card, StatusDot, SeverityBadge: Shared UI primitive components with consistent design system.
- CampusMap: SVG-based campus visualization with zone risk coloring, incident dot overlay, geofence animation, and live indicator (FR-02, FR-09).
- PipelineView: Four-stage verification pipeline visualization with AI confidence bar, NFR-08 gate, incident metadata grid, and action buttons (FR-03, FR-04, FR-05, NFR-08).
- CSAS (main export): Root component managing all application state via React hooks; implements submitReport, verify, reject, confirm, triggerSOS, and runSimulation action handlers.
- Eight page views: dashboard, report, map, verification, analytics, log, simulation, architecture — each traceable to one or more FRs/NFRs.

index.css — Tailwind CSS base configuration with dark background (#0a0f1a) applied globally.

main.jsx — React DOM root mount point, rendering the CSAS component into the #root element with StrictMode enabled.

Repository structure follows the Workshop No. 3 CI/CD discipline: source files are organized in the course GitHub repository under the CourseProject/ folder alongside the technical report PDF, workshop deliverables, and simulation scripts.

A RAW SURVEY DATA (WORKSHOP NO. 1)

Table A1 presents the complete raw data from the student survey (n = 20) conducted during Workshop No. 1.

Table A1. Raw Survey Data (n = 20). Variables: safety perception, incident experience, incident type, app adoption intention, and time of day.

ID	Feels Safe	Experienced	Incident Type	Would Use App	Time
1	No	Yes	Theft	Yes	Night
2	Yes	No	None	Yes	Afternoon
3	No	Yes	Harassment	Yes	Night
4	Yes	No	None	Maybe	Morning
5	Yes	No	None	Maybe	Morning
6	Yes	No	None	Yes	Afternoon
7	No	Yes	Theft	Yes	Night
8	Yes	No	None	Maybe	Morning
9	Yes	No	None	Maybe	Morning
10	Yes	No	None	Yes	Afternoon
11	No	Yes	Suspicious Activity	Yes	Night
12	Yes	No	None	Maybe	Morning
13	No	Yes	Theft	Yes	Evening
14	Yes	No	None	Yes	Afternoon
15	No	Yes	Harassment	Yes	Night
16	Yes	No	None	Maybe	Morning
17	Yes	No	None	Maybe	Night
18	Yes	No	None	Yes	Afternoon
19	No	Yes	Theft	Yes	Night
20	Yes	No	None	Maybe	Morning

Experienced incident: Yes: 7 (35%) — No: 13 (65%). All reported incidents occurred during afternoon or night hours.

B DIRECT OBSERVATION RECORD (WORKSHOP NO. 1)

Table B1 presents the complete direct observation record from the eight sessions conducted across four campus zones during Workshop No. 1.

Table B1. Direct Observation Record — 8 sessions across 4 campus zones.

Session	Location	Time	Activity Level	Suspicious Event
1	Main Entrance	Morning	High	No
2	Library Area	Afternoon	High	No
3	Parking Lot	Night	Medium	Yes
4	Dormitory Area	Evening	Medium	No
5	Main Entrance	Afternoon	High	No
6	Parking Lot	Night	Low	Yes
7	Library Area	Morning	Medium	No
8	Dormitory Area	Evening	Medium	No

C BENCHMARKING OF EXISTING SECURITY PLATFORMS

Table C1 presents the benchmarking comparison of three existing security alert platforms operating in Bogotá conducted during Workshop No. 1.

Table C1. Benchmarking of Existing Security Alert Platforms in Bogotá.

Platform	Strengths	Weaknesses	Insight for CSAS
Policía Nacional / Vigila App	Direct police integration; national coverage	Slow response; requires a phone call	Authority integration model is valuable but must be faster and mobile-first.
Neighborhood networks (WhatsApp)	High adoption; instant notifications; familiar channel	No verification; no records; generates panic	Immediacy is a key design goal, but verification is non-negotiable.
SIURE UD (internal)	Exists; staff familiar; institutional endorsement	Email and phone only; no mobile or real-time capability	CSAS complements rather than replaces SIURE UD, building on institutional familiarity.